

CP HW07

2025-12-29

Aksel Shen

20234001053@m.scnu.edu.cn
South China Normal University

1

考虑如下类似 LISP 语言的前缀表达式文法：

$$\begin{aligned}\text{lexp} &\rightarrow \text{number} \mid (\text{op lexp-seq}) \\ \text{op} &\rightarrow + \mid - \mid * \\ \text{lexp-seq} &\rightarrow \text{lexp-seq lexp} \mid \text{lexp}\end{aligned}$$

定义一个语义属性 val 为表达式的值。例如，对于前缀表达式 $(* (-2) 3 4)$ ，即 $(-2) \times 3 \times 4$ 的值 $\text{val} = -24$ 。

- (1) 请为上述文法写出相应的语义规则。假设对于非终结符 **number** 的属性值可由函数 $\text{getval}()$ 得到。
- (2) 请给出输入 $(* (-2) 3 4)$ 所对应的包含属性计算的语法树，并标出计算的次序。

解

(1) 语义规则如下：

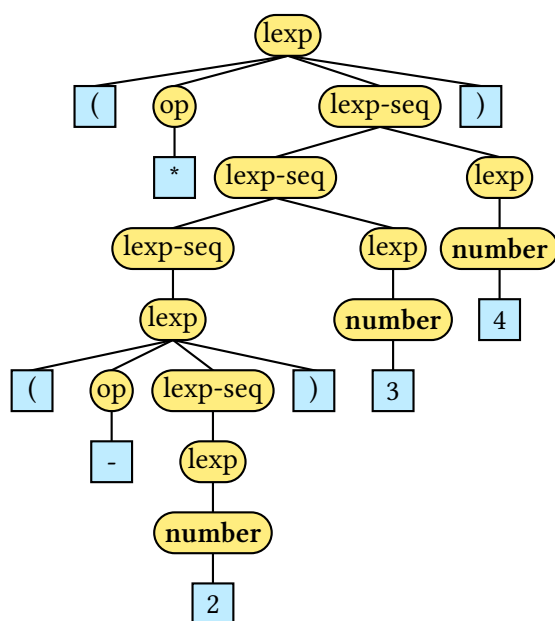
Production	Semantic Rule
$\text{lexp} \rightarrow \text{number}$	$\text{lexp.val} = \text{getval}(\text{number})$
$\text{lexp} \rightarrow (\text{op lexp-seq})$	$\text{lexp.val} = \text{apply}(\text{op.type}, \text{lexp-seq.vals})$
$\text{op} \rightarrow +$	$\text{op.type} = \text{Add}$
$\text{op} \rightarrow -$	$\text{op.type} = \text{Sub}$
$\text{op} \rightarrow *$	$\text{op.type} = \text{Mul}$
$\text{lexp-seq} \rightarrow \text{lexp}$	$\text{lexp-seq.vals} = [\text{lexp.val}]$
$\text{lexp-seq} \rightarrow \text{lexp-seq}_1 \text{ lexp}$	$\text{lexp-seq.val} = \text{append}(\text{lexp-seq}_1.\text{vals}, \text{lexp.val})$

其中 vals 是一个综合属性，表示表达式序列的值列表；函数 $\text{append}(a, b)$ 表示将值 b 添加到列表 a 的末尾；函数 $\text{apply}(\text{op}, a)$ 表示对列表 a 中的值按操作符 op 进行计算。

(2) 计算顺序：

- (1) $\text{val} = \text{getval}(2) = 2$.
- (2) $\text{lexp-seq.vals} = [2]$.
- (3) $\text{op.type} = \text{Sub}$.
- (4) $\text{apply}(\text{Sub}, [2]) = -2$.
- (5) $\text{lexp-seq.vals} = [-2]$.
- (6) $\text{val} = \text{getval}(3) = 3$.
- (7) $\text{lexp-seq.vals} = \text{append}([-2], 3) = [-2, 3]$.
- (8) $\text{val} = \text{getval}(4) = 4$.
- (9) $\text{lexp-seq.vals} = \text{append}([-2, 3], 4) = [-2, 3, 4]$.
- (10) $\text{op.type} = \text{Mul}$.
- (11) $\text{apply}(\text{Mul}, [-2, 3, 4]) = (-2) \times 3 \times 4 = -24$.

语法树：



2

考虑如下的一段三地址中间代码：

```
01: dp = 0
02: i = 0
03: t1 = i * 8
04: t2 = A[t1]
05: t3 = i * 8
06: t4 = B[t3]
07: t5 = t2 * t4
08: dp = dp + t5
09: i = i + 1
10: if (i < n) goto (03)
```

- (1) 请将上述代码划分为基本块，并画出流图。
- (2) 请使用消除公共子表达式，对归纳变量进行强度消减，消除归纳变量等方法，尽可能优化上述代码。请写出优化的过程和依据。

解

(1) 基本块划分

由于代码在 10: if (i < n) goto (03) 处有条件跳转，且没有其他跳转指令，因此可以从跳转目标 03 开始划分出一个新的基本块，如下：

• B_1 (入口块)：

```
01: dp = 0
02: i = 0
```

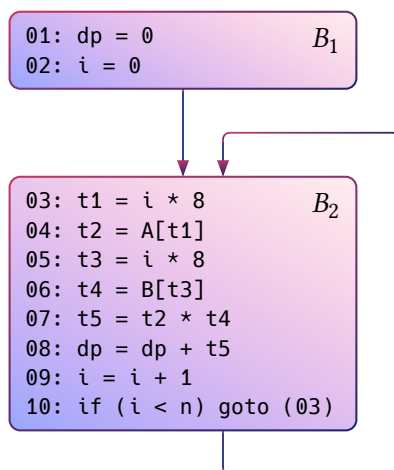
- B_2 (循环体) :

```

03: t1 = i * 8
04: t2 = A[t1]
05: t3 = i * 8
06: t4 = B[t3]
07: t5 = t2 * t4
08: dp = dp + t5
09: i = i + 1
10: if (i < n) goto (03)

```

流图:



(2) 代码优化:

(a) 消除公共子表达式

在基本块 B_2 中, 语句 03 和 05 计算相同的表达式 $i * 8$:

```

03: t1 = i * 8
05: t3 = i * 8

```

可以消除重复计算, 将 t_3 替换为 t_1 :

```

03: t1 = i * 8
04: t2 = A[t1]
06: t4 = B[t1]    // Replace t3 with t1
07: t5 = t2 * t4
08: dp = dp + t5
09: i = i + 1
10: if (i < n) goto (03)

```

(b) **Strength Reduction**

归纳变量分析:

- i 是基本归纳变量, 在循环中每次加 1
- $t_1 = i * 8$ 是派生归纳变量, 依赖于 i

可以将乘法操作 $i * 8$ 替换为加法操作。引入新变量 t_{1_new} :

- 初始值: $t_{1_new} = 0$ (因为 i 初始为 0)
- 每次迭代: $t_{1_new} = t_{1_new} + 8$

优化后的代码：

```
01: dp = 0
02: i = 0
02.5: t1 = 0          // Initialization
03: t2 = A[t1]        // Delete t1 = i * 8
04: t4 = B[t1]
05: t5 = t2 * t4
06: dp = dp + t5
07: i = i + 1
08: t1 = t1 + 8        // Strength Reduction
09: if (i < n) goto (03)
```

(3) 删除无用变量

变量 i 在循环体内只用于循环条件判断和递增。经过强度消减后， $i * 8$ 已被 $t1$ 的增量更新替代。但 i 仍需用于循环终止条件 $i < n$ 。

如果可以将终止条件改为用 $t1$ 表示 ($t1 < n * 8$)，则可以完全消除 i ：

```
01: dp = 0
02: t1 = 0
03: t2 = A[t1]
04: t4 = B[t1]
05: t5 = t2 * t4
06: dp = dp + t5
07: t1 = t1 + 8
08: if (t1 < n * 8) goto (03)
```

进一步优化，可以预先计算 $n * 8$ ：

```
01: dp = 0
02: t1 = 0
02.5: limit = n * 8    // 预计算循环上界
03: t2 = A[t1]
04: t4 = B[t1]
05: t5 = t2 * t4
06: dp = dp + t5
07: t1 = t1 + 8
08: if (t1 < limit) goto (03)
```

最终优化结果：

```
01: dp = 0
02: t1 = 0
03: limit = n * 8
04: t2 = A[t1]
05: t4 = B[t1]
06: t5 = t2 * t4
07: dp = dp + t5
08: t1 = t1 + 8
09: if (t1 < limit) goto (04)
```